BS Boris Shwartz
April 14 2025 | Updated August 30 2025

Kubernetes Services Explained: Beginner's Guide to Cluster Communication

Kubernetes

In the previous article Step-by-Step Guide to Deploy and Manage Kubernetes, we covered the fundamentals of Kubernetes: its architecture, key components (such as Pods, Deployments, and Services), and general principles for cluster deployment and management. Now, we'll dive deeper into working with **Pods**—the smallest executable units in Kubernetes. To effectively manage their communication and availability, Kubernetes uses **Services**, whose general purpose was mentioned earlier.

Linux VPS Servers
Deploy a Cloud Linux Server
with Ubuntu, CentOS, Debian, or
Oracle Linux preset

In this article, we'll explore the functionality of Services through practical examples: their types (ClusterIP, NodePort, LoadBalancer), YAML manifest configurations, and use cases for ensuring stable communication between application components.

Kubernetes Services Revisited

First, let's recall what **Kubernetes Services** are. In short, they are objects that provide stable access to applications in a cluster, even when Pods change dynamically. More comprehensively, **Kubernetes Services** are a critical mechanism for ensuring stable network communication between application components in a dynamically changing cluster environment. Simply put, Services act as **"stable entry points"** to a group of Pods, even if the

Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF

Pods themselves are recreated, scaled, or moved between nodes. Below is an illustration of the main types of Services in K8s:

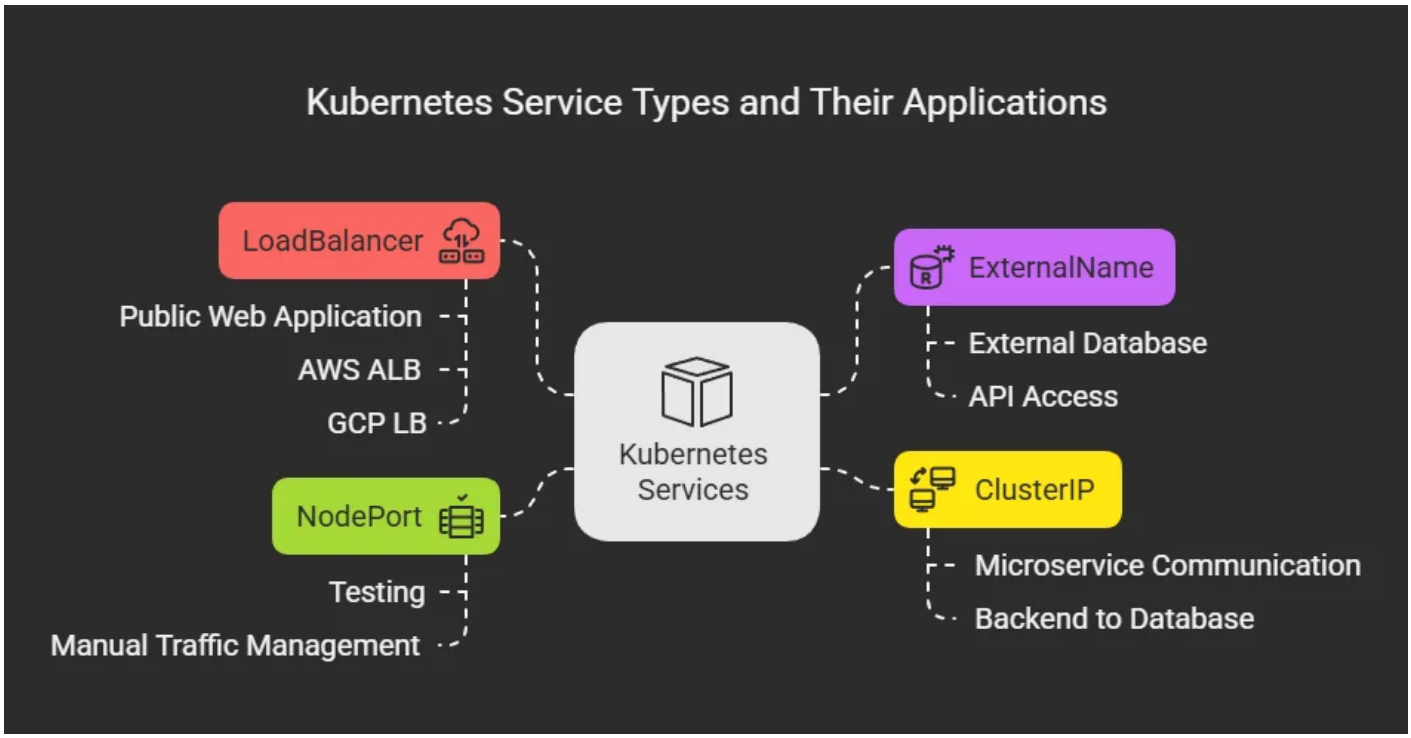

Image 1 - K8s services

Problems Addressed by Kubernetes Services:

Since Pods in Kubernetes are ephemeral—their IP addresses change during restarts, scaling, or updates—this creates challenges for application components (e.g., frontend and backend) that require a stable communication method.

Kubernetes Services solve these issues through the following actions:
- **Fixed Identifiers**: Services assign a stable IP address (ClusterIP), DNS name, or external IP, independent of the Pods' lifecycle.
- **Load Balancing**: Traffic is automatically distributed across Pods selected via labels (e.g., àpp: backend).
- **Auto-Updating**: Services dynamically track changes in the set of Pods via EndpointSlices, updating the list of available instances.

Core Kubernetes Service Types with Examples

ClusterIP: For Internal Access to Pods Within the Cluster

Example configuration:

Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF

```
simple-service.yaml
apiVersion: v1
kind: Service
metadata:
name: my-service
spec:
selector:
app.kubernetes.io/name: MyApp
ports:
- protocol: TCP
port: 80
targetPort: 9376
```

Command to create the Service:

```
kubectl apply -f simple-service.yaml
```

Checking:

```
kubectl get services # Viewing ClusterIP
kubectl describe service/my-service # Details Service and Enc
```

LoadBalancer is designed for public access through a cloud load balancer.

Example configuration:

```
# nodeport-service.yaml
apiVersion: v1
kind: Service
```

Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF

```
metadata:
name: my-nodeport-service
spec:
type: NodePort
selector:
app.kubernetes.io/name: MyApp
ports:
- port: 80
targetPort: 9376
nodePort: 30007 # Optional (default: random port in range)

```

Command to create the Service:

```
kubectl apply -f nodeport-service.yaml
```

Access:

```
curl http://:30007 # replace  with node IP
```

ExternalName is designed to link to an external resource via DNS CNAME.

Example configuration:

```
# loadbalancer-service.yaml
apiVersion: v1
kind: Service
metadata:
name: my-loadbalancer
spec:
type: LoadBalancer
```

Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF

```
selector:
app.kubernetes.io/name: MyApp
ports:
- port: 80
targetPort: 9376
```

Command to create the Service:

```
kubectl apply -f loadbalancer-service.yaml
```

Checking the external IP :

```
kubectl get service/my-loadbalancer -o wide # Viewing EXTERNA
```

Headless Service (without ClusterIP) is designed for direct access to Pods without load balancing.

Example configuration:

```
# externalname-service.yaml
apiVersion: v1
kind: Service
metadata:
name: external-db
spec:
type: ExternalName
externalName: my.database.example.com
```

Command to create the Service:

Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF

```
kubectl apply -f externalname-service.yaml
```

Usage:
Other Pods can access èxternal-db`as a DNS name

Headless Service (without ClusterIP) is designed for direct access to Pods without load balancing.

Configuration exmaple:

```
# headless-service.yaml
apiVersion: v1
kind: Service
metadata:
name: my-headless
spec:
clusterIP: None
selector:
app.kubernetes.io/name: MyApp
ports:
- port: 80
targetPort: 9376

```

Command to create the Service:

```
kubectl apply -f headless-service.yaml
```

DNS checking:

```
nslookup my-headless.default.svc.cluster.local # Returns the
```

Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF

Additional examples:

Multi-port Service configuration is designed to simplify setup for applications requiring multiple ports.

Configuration:

```yaml
# multi-port-service.yaml
apiVersion: v1
kind: Service
metadata:
name: my-multi-port
spec:
selector:
app.kubernetes.io/name: MyApp
ports:
- name: http
protocol: TCP
port: 80
targetPort: 8080
- name: metrics
protocol: TCP
port: 9090
targetPort: 9090
```

Command:

```
kubectl apply -f multi-port-service.yaml
```

Service configuration without a selector (manual EndpointSlices) is used to integrate with resources outside Kubernetes

Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF

Service configuration:

```yaml
# service-without-selector.yaml
apiVersion: v1
kind: Service
metadata:
name: my-external-service
spec:
ports:
- name: http
protocol: TCP
port: 80
targetPort: 9376

```

EndpointSlice configuration is designed for flexible management of endpoints (including external systems):

Configuration:

```yaml
# endpointslice.yaml
apiVersion: discovery.k8s.io/v1
kind: EndpointSlice
metadata:
name: my-service-1
labels:
kubernetes.io/service-name: my-external-service
addressType: IPv4
ports:
- name: http
protocol: TCP
port: 9376
endpoints:

```

Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF

```
- addresses: ["10.4.5.6"]
- addresses: ["10.1.2.3"]
```

Commands:

```
kubectl apply -f service-without-selector.yaml
```

```
kubectl apply -f endpointslice.yaml
```

Useful commands:

Viewing all Services is used to display a list of all Services in the current namespace:

```
kubectl get services
```

Viewing Endpoints is intended to display a list of Endpoints – the IP addresses and ports of Pods associated with the Service:

```
kubectl get endpoints
```

DNS verification for a Service is intended to check the DNS resolution of the Service name within a Pod:

```
kubectl exec -it  -- nslookup
```

Deletion of a Service is intended to remove the Service from the cluster:

Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF

```
kubectl delete service/
```

FAQ: Kubernetes Services

- **Q1: What is a Kubernetes Service?**
  A1: A Kubernetes Service is an abstraction that provides stable network access to a set of Pods. Services ensure reliable communication between components, even when Pods are recreated, scaled, or moved between nodes.
- **Q2: Why are Services needed in Kubernetes?**
  A2: Pods are ephemeral, meaning their IP addresses can change during restarts or scaling. Services solve this by providing fixed IPs, DNS names, or external addresses, along with load balancing and automatic updates of endpoints.
- **Q3: What are the main types of Services in Kubernetes?**
  A3: Core types include:
  ClusterIP - internal access within the cluster
  NodePort - exposes the service on a static port on each node
  LoadBalancer - integrates with cloud provider load balancers for external access
  ExternalName - maps a service to an external DNS name
  Headless Service - direct Pod access without load balancing
- **Q4: How do I create a Kubernetes Service?**
  A4: Services are defined in YAML manifests and applied using kubectl apply -f <filename.yaml>. You specify the type, selector (labels), and ports to configure how traffic is routed to Pods.
- **Q5: Can a Service have multiple ports?**
  A5: Yes. Multi-port Services allow exposing several ports from the same set of Pods, useful for applications that serve HTTP, metrics, and other protocols simultaneously.
- **Q6: How can I verify that a Service is working?**
  A6: Use commands like kubectl get services to list Services, kubectl describe service/<name> to view details, and nslookup <service-name> inside a Pod to check DNS resolution.
- **Q7: What if I need to connect a Service to external endpoints?**
  A7: You can create a Service without a selector and manually define endpoints via EndpointSlice objects, allowing integration with resources outside the Kubernetes cluster.
- **Q8: How do Services handle load balancing?**
  A8: Services automatically distribute incoming traffic among the selected Pods based on labels, ensuring even load and high availability within the cluster.

Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF

Vote: ★ ★ ★ ★ ★ 4 out of 5

Average rating : 4.5

Rated by: 2

You might also like...

CUSTOMER'S CHOICE

Linux VPS Server

Rent a Linux-based virtual server from €4/mo

Order Now

Step-by-Step Guide to Deploy and Manage Kubernetes Clusters for Beginners

Learn how to deploy and manage Kubernetes clusters with this step-by-step guide. Perfect for beginners, it covers Kubernetes basics, core concepts, and essential kubectl commands to help

Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF

Read more

How to install Jenkins on Kubernetes

Jenkins is a continuous integration server that is distributed open source in the Java language.

Read more

Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF